

Эмбеддинги. Word2vec, fasttext

Угадай слово по контексту

«... наладил ____ с ключевыми клиентами, провёл 12 встреч»

«за ____ отвечает руководитель отдела продаж»

«еженедельный ____: лиды, конверсия, выручка»

Представления слов

Слова представляются в виде one-hot вектора:

умный	$[0, \dots, 1, \dots, 0, \dots, 0, \dots, 0]$
сообразительный	$[0, \dots, 0, \dots, 1, \dots, 0, \dots, 0]$
собака	$[0, \dots, 0, \dots, 0, \dots, 1, \dots, 0]$

Недостатки:

- Большая размерность векторов;
- Все вектора одинаково похожи;
- Сложно кодировать дополнительную информацию.

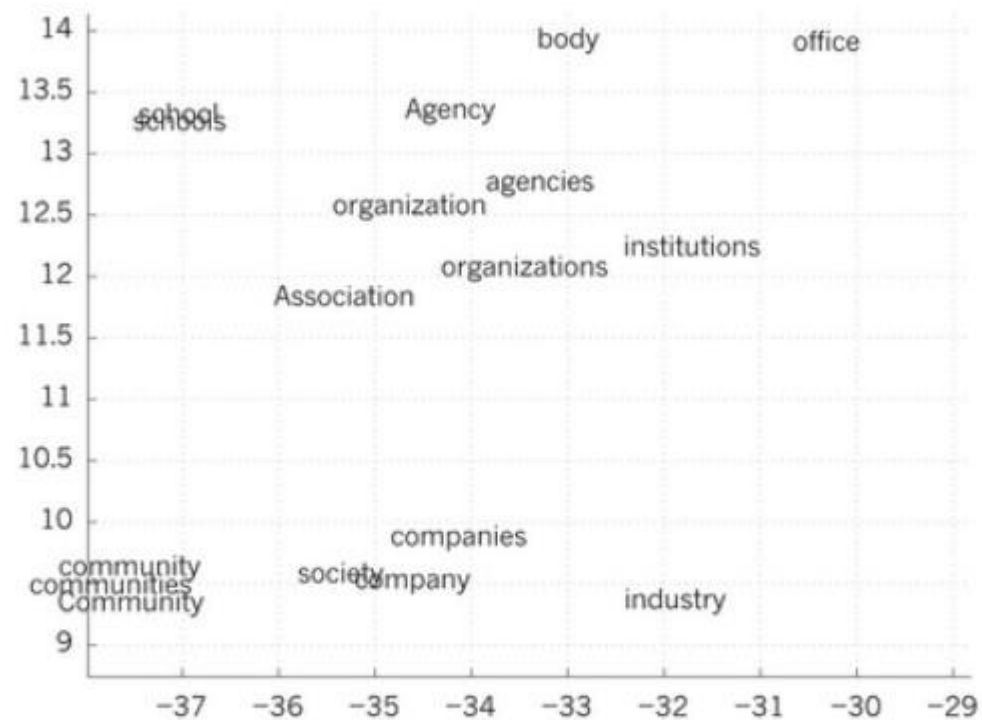
Гипотеза компактности

Бликие (похожие) слова встречаются в похожем контексте.

Идея: у близких слов (синонимов, родственных слов) и представления получают близкие.

Эмбединги. Word2vec, fasttext

Векторное представление слова (word embedding) — вещественный вектор в пространстве с фиксированной невысокой размерностью.



Зачем нужны Word embeddings?

Сжатые векторные представления слов

1. полезны сами по себе, например, для поиска синонимов или опечаток в поисковых запросах.
2. используются в качестве признаков для решения самых различных задач: выявление именованных сущностей, тэгирование частей речи, машинный перевод, кластеризация документов, ранжирование документов, анализ тональности текста.

word2vec

Мера семантической близости — мера близости, предназначенная для количественной оценки семантической схожести слов. Такая мера показывает высокие значения для пар слов, находящихся в семантических отношениях (синонимия, ассоциативность и т.д.), и нулевые значения для всех остальных пар.

word2vec - алгоритм для получения векторных представлений слов. Подход основан на важной гипотезе, которую в науке принято называть гипотезой локальности — “слова, которые встречаются в одинаковых окружениях, имеют близкие значения”. Близость в данном случае понимается очень широко, как то, что рядом могут стоять только сочетающиеся слова. Например, для нас привычно словосочетание "заводной будильник". А сказать “заводной апельсин” мы не можем* — эти слова не сочетаются.

Алгоритм word2vec

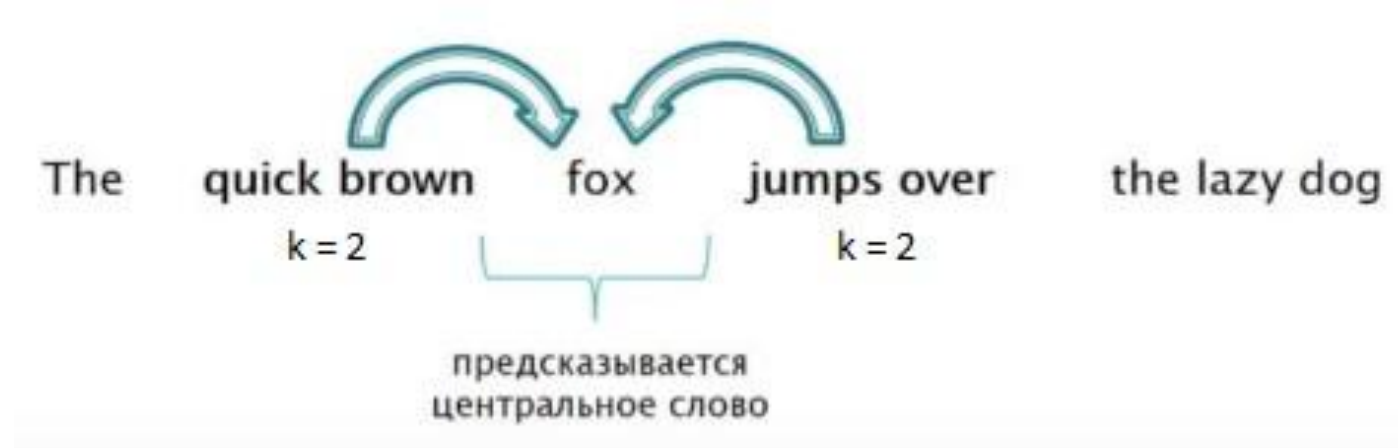
Мы будем предсказывать вероятность слова по его окружению (контексту). То есть мы будем учить такие вектора слов, чтобы вероятность, присваиваемая моделью слову была близка к вероятности встретить это слово в этом окружении в реальном тексте.

$$P(w_o | w_c) = \frac{e^{s(w_o, w_c)}}{\sum_{w_i \in W} e^{s(w_i, w_c)}}$$

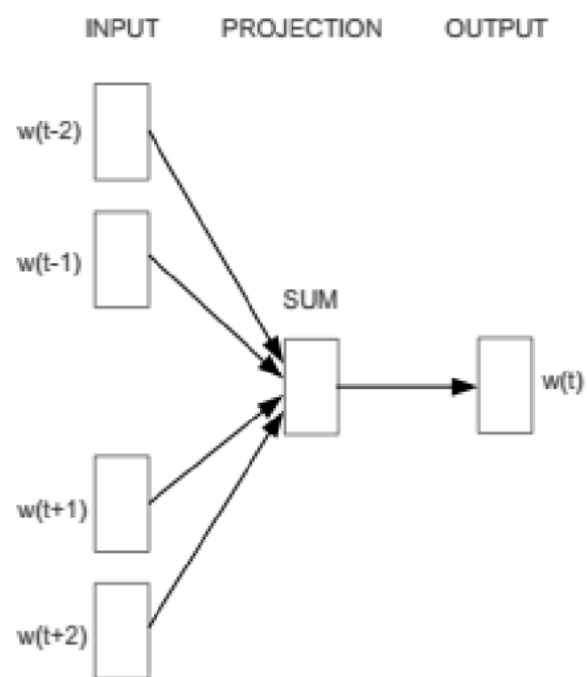
Здесь w_o — вектор целевого слова, w_c — это некоторый вектор контекста, вычисленный (например, путем усреднения) из векторов окружающих нужное слово других слов. А S — это функция, которая двум векторам сопоставляет одно число. Например, это может быть косинусное расстояние.

CBOW

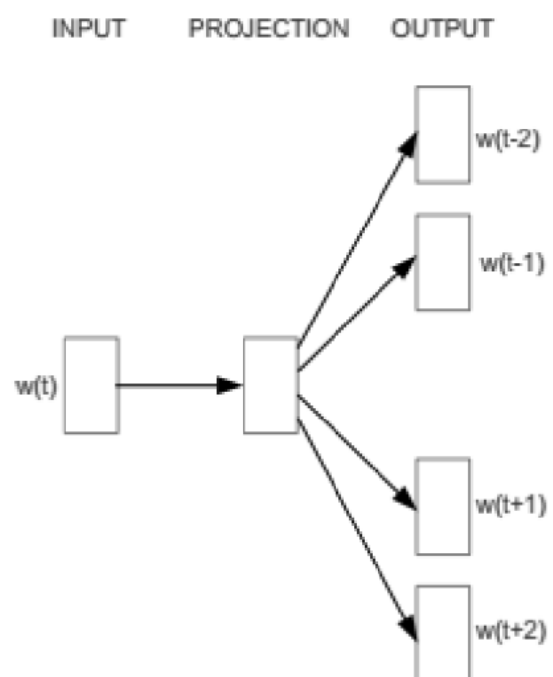
Процесс тренировки устроен следующим образом: мы берем последовательно $(2k+1)$ слов, слово в центре является тем словом, которое должно быть предсказано. А окружающие слова являются контекстом длины по k с каждой стороны. Каждому слову в нашей модели сопоставлен уникальный вектор, который мы меняем в процессе обучения нашей модели. В целом, этот подход называется CBOW — continuous bag of words, continuous потому, что мы скормливаем нашей модели последовательно наборы слов из текста, а BoW потому что порядок слов в контексте не важен.



Word2Vec

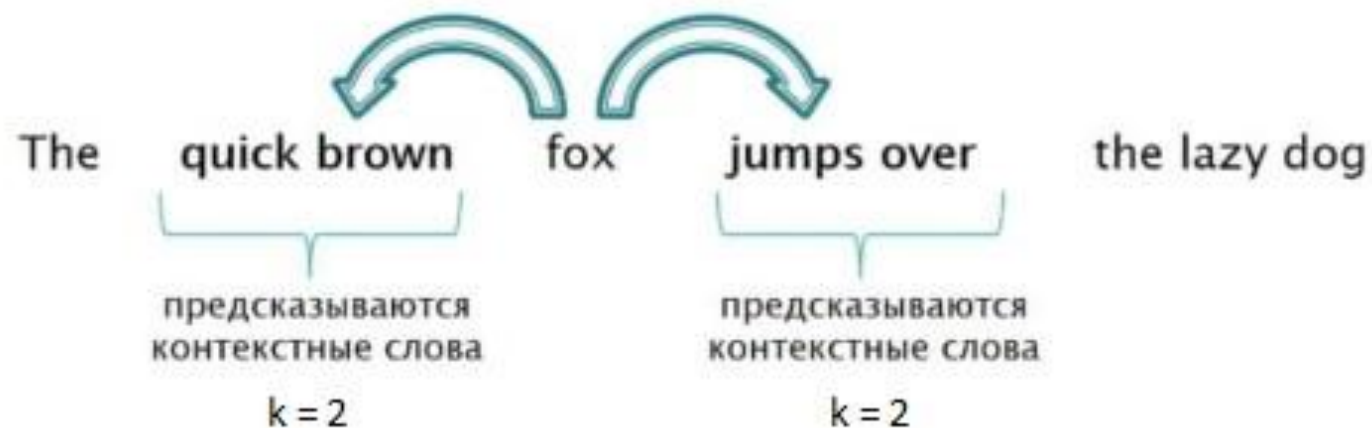


CBOW



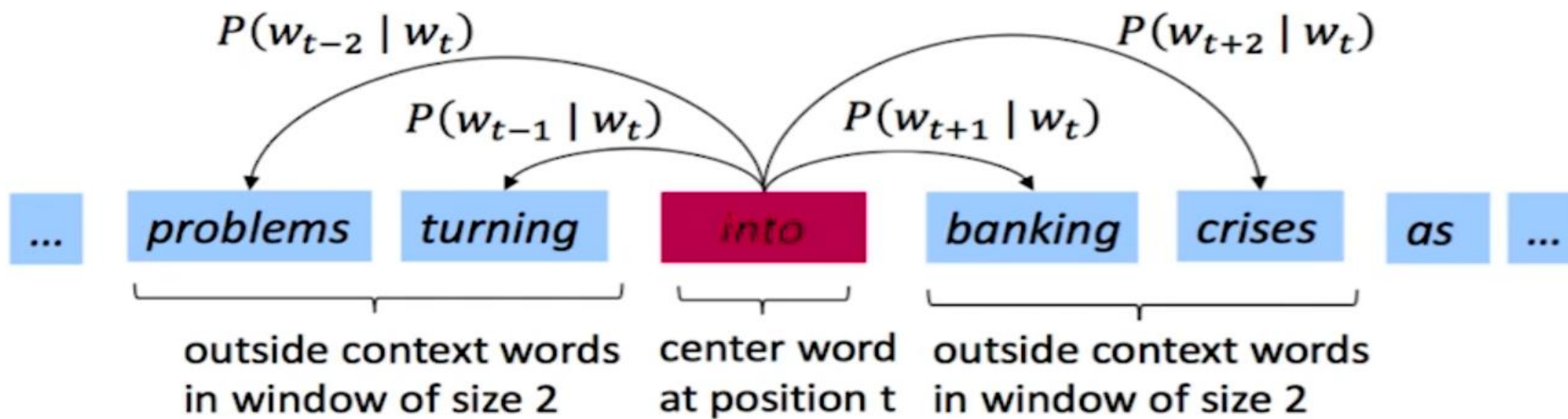
Skip-gram

Другой подход skip-gram — прямо противоположный CBOW, то есть “словосочетание с пропуском”. Мы пытаемся из данного нам слова угадать его контекст (точнее вектор контекста). В остальном модель не претерпевает изменений.



Skip-gram

Зная контекст, мы хотим максимизировать вероятность центрального слова. Перемещаемся скользящим окном.



Skip-gram

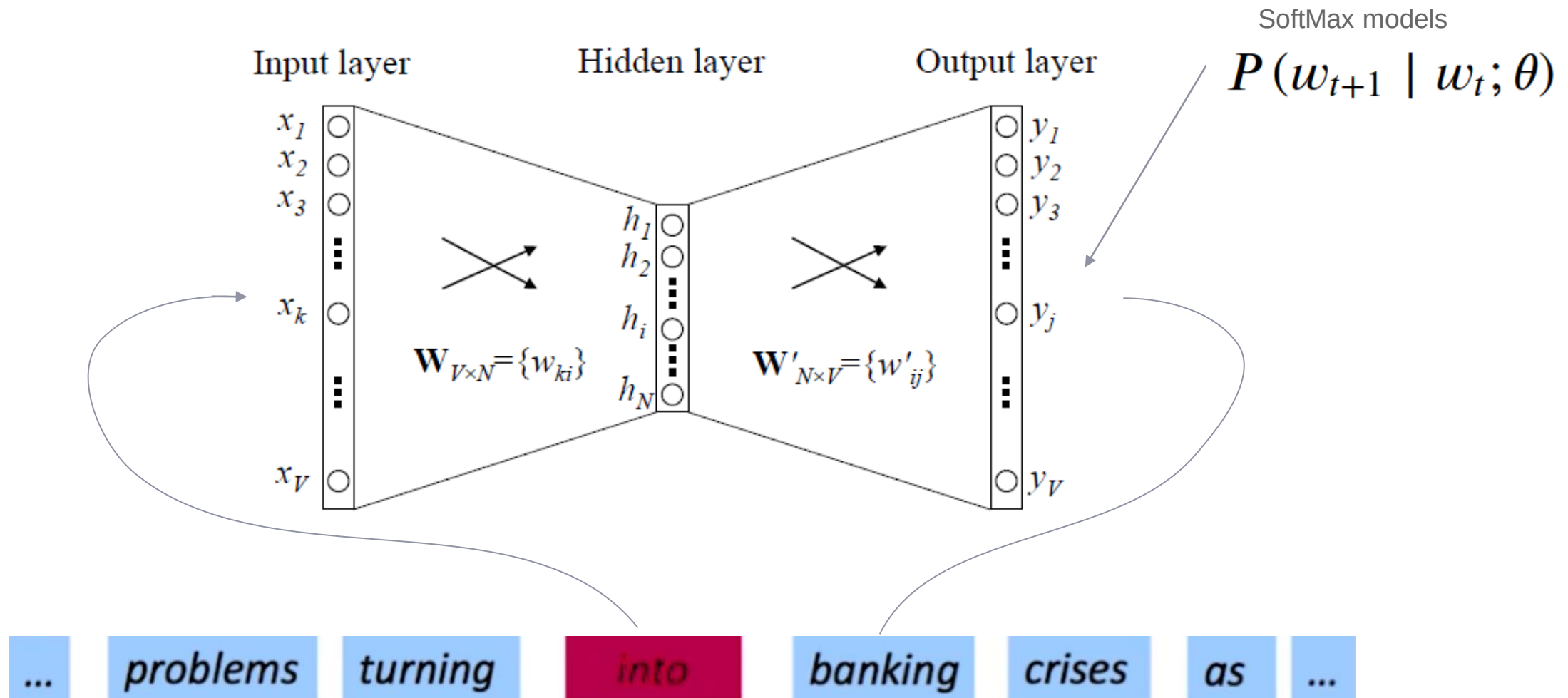
Вероятность встретить слово w_o рядом со словом w_I :

$$p(w_o|w_I) = \frac{\exp(\langle v'_{w_o}, v_{w_I} \rangle)}{\sum_{w \in W} \exp(\langle v'_w, v_{w_I} \rangle)}$$

Функционал для текста $T=(w_1 w_2 \dots w_n)$:

$$\sum_{i=1}^n \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log p(w_{i+j}|w_i) \rightarrow \max$$

Skip-gram



Skip-gram

Простейшая модель классификации — линейный классификатор, применяемый к вектору контекста.

$$\textit{softmax}(\boldsymbol{x})_i = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} = p_i$$

Для обучения минимизируем функцию потерь.

Skip-gram

Вероятность встретить слово w_o рядом со словом w_I :

$$p(w_o|w_I) = \frac{\exp(\langle v'_{w_o}, v_{w_I} \rangle)}{\sum_{w \in W} \exp(\langle v'_w, v_{w_I} \rangle)}$$

Считать знаменатель крайне затратно. Значит, и производные считать тоже долго.

Negative sampling

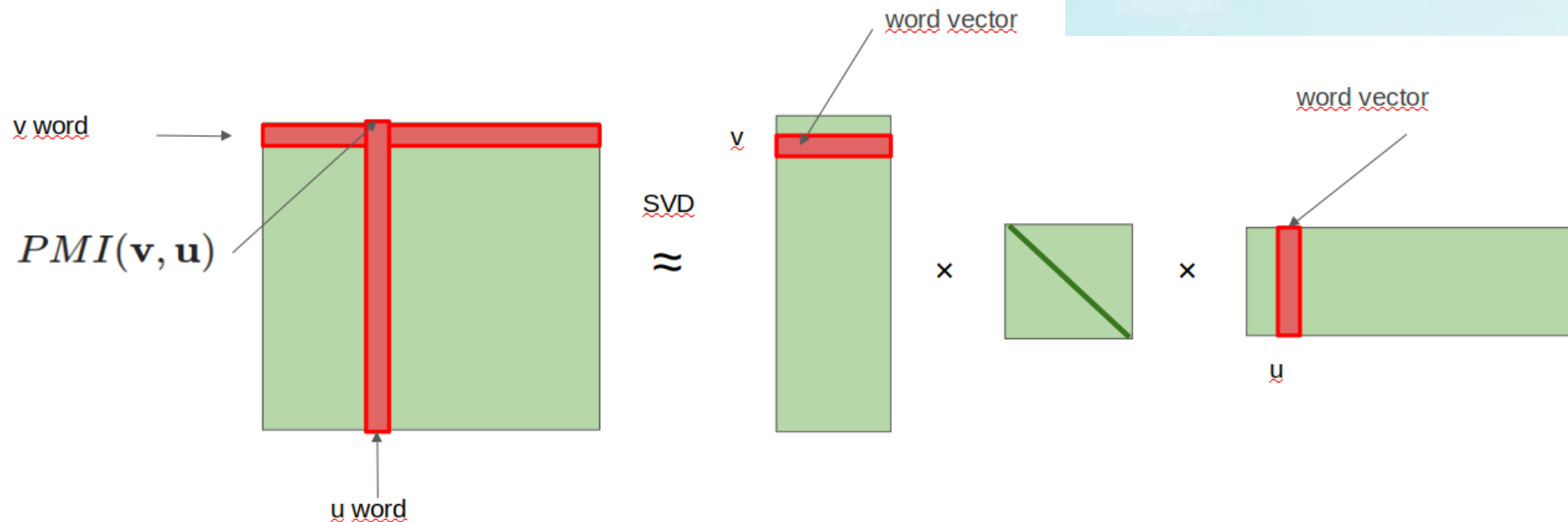
$$p(w_o | w_I) = \log \sigma(\langle v'_{w_o}, v_{w_I} \rangle) + \sum_{i=1}^k \log \sigma(-\langle v'_{w_i}, v_{w_I} \rangle)$$

w_i — случайно выбранные слова. Слово w генерируется с вероятностью $P(w)$ — шумовое распределение.

$$P(w) = \frac{U(w)^{\frac{3}{4}}}{\sum_{v \in W} U(v)^{\frac{3}{4}}}, U(v) \text{ — частота слова } v \text{ в корпусе текстов.}$$

Word2vec vs SVD

Word2Vec with negative sampling $\approx$ matrix factorization



- Один из вариантов оценки — PMI (*Pointwise Mutual Information*):

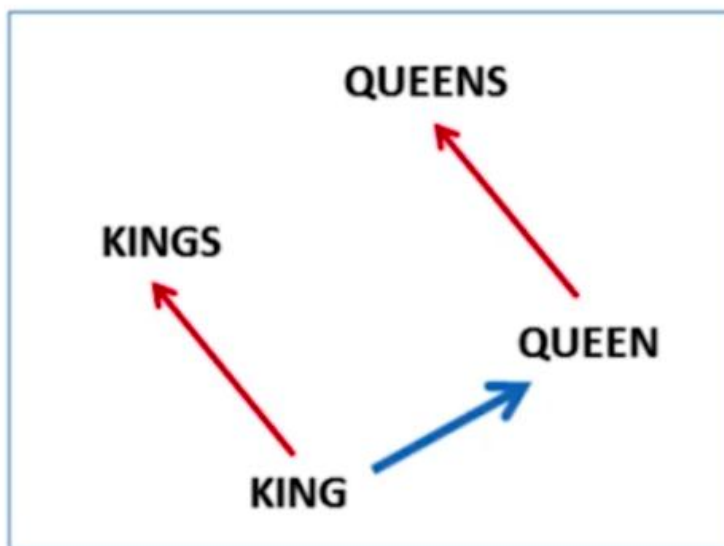
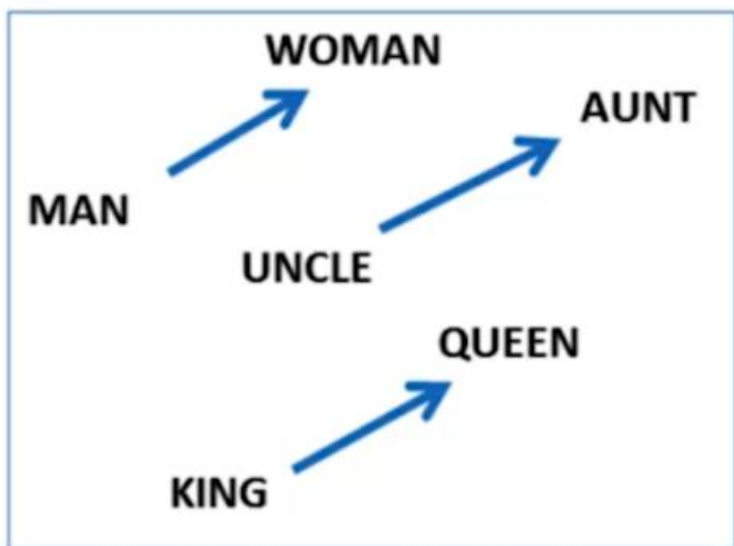
$$PMI(u, v) = \log \frac{N_{uv}}{N_u N_v}$$

- N_{uv} — число документов, содержащих оба слова u и v в окне заданного размера
- N_u — число документов, содержащих слово u

метод сингулярного разложения (Singular Value Decomposition, SVD)

Word2vec

$$\underline{v(\text{king})} - \underline{v(\text{man})} + \underline{v(\text{woman})} \approx \underline{v(\text{queen})}$$



- Москва – Россия + Франция ≈ Париж»
- «руководитель – продажи + маркетинг ≈ head_of_marketing»

Слово "мужчина" относится к слову "женщина" так же, как слово "дядя" к слову "тётя", что для нас совершенно естественно и понятно, но в других моделях добиться такого же соотношения векторов можно только с помощью специальных ухищрений. Здесь же — это происходит естественно из самого корпуса текстов.

Fasttext

Проблемы Word2Vec:

OOV (out-of-vocabulary): новое/редкое слово не имеет вектора.

Морфология: «маркетолог», «маркетологи», «маркетинговый» — три разные формы → три разреженные вектора.

Разделите слово на мешок из n -граммов: *apple* = $\langle ap, ppl, ple, le \rangle$ (BPE).

Вычислите вектор для каждого n -грамма.

Вектор для слова = сумма <вектора слова, векторов для n -граммов слов>.

Fasttext

Слово «клиент» → добавим маркеры границ: `<клиент>`.

Пусть `minn=3`, `maxn=6`.

3-граммы:

`<кл , кли , лие , иен , ент , нт>`

4-граммы:

`<кли , клие , лиен , иент , ент>`

5-граммы:

`<клие , клиен , лиент , иент>`

6-граммы:

`<клиен , клиент , лиент>`

- специальный токен всего слова `"<клиент>"` (как целое).

Вектор слова w — среднее/сумма векторов его подслов:

$$\mathbf{z}_w = \frac{1}{|G_w|} \sum_{g \in G_w} \mathbf{z}_g,$$

где G_w — множество n -грамм (с учётом границ).

Новое слово – клиентоориентированность.
Откуда возьмется вектор?

Гиперпараметры Word2Vec fastText

model: "skipgram" (лучше на редких словах) или "cbow" (быстрее на больших).

dim: 100–300 (малые корпуса — 100–200; крупные — 300+).

WS (window): 3–5 для синтаксики, 8–10 для тематики.

epoch: 5–15.

minCount: 5–20.

negative (или по умолчанию в native fastText): 5–15.

lr: 0.025–0.05.

Ключевые гиперпараметры fastText

minn/maxn: границы длины символьных n-грамм (типично 3–6).

bucket: число хеш-корзин для n-грамм (например 2e6).

Ваши вопросы?

Классические подходы к обработке естественного языка, урок 3